

Устройство памяти в Python

Управление памятью

Управление памятью — это процесс выделения, распределения и координации памяти таким образом, чтобы все программы работали правильно и могли оптимально получать доступ к различным системным ресурсам.

Управление памятью также включает в себя и очистку памяти от объектов, которые больше не нужны.

Типы управления памятью:

- ☐ ручное
- ☐ автоматическое

Ручное управление памятью

Ручное управление памятью включает в себя, как правило, три этапа:

- ☐ запрос памяти у операционной системы
- ☐ работа с ней
- ☐ возвращение памяти обратно в операционную систему

Ручное управление памятью

Преимущества:

Ручной подход управления позволяет работать с памятью максимально эффективно. Точно известно, сколько памяти выделено, зачем мы ее используем, и т.д.

Недостатки:

Сложность. Управлять памятью вручную — сложно и тяжело, поскольку если не вернуть память обратно операционной системе, возникает утечка: программа держит неиспользуемую память, не дает применять ее для решения других задач.

Автоматическое управление памятью

Автоматическое управление памятью берет на себя самый сложный этап — возвращение памяти обратно операционной системе, когда она уже не требуется.

Восстановленная память может использоваться другими объектами. Это немного менее эффективно, но позволяет сократить трудозатраты на управление памятью и повысить надежность этого процесса.

К языкам с ручным управлением памяти относятся: **C, C++, Pascal** и др.

К языкам с автоматическим управлением памяти относятся: **Python, C#, Java, Ruby, JavaScript** и др.

Механизм памяти в Python

Для управления памятью Python использует несколько механизмов.

При запуске Python программы создается **новый процесс**, в рамках которого операционная система выделяет **пул ресурсов**, включая **виртуальное адресное пространство**.

В эту память загружается интерпретатор Python вместе со всеми необходимыми ему для работы данными, включая код программы.

Механизм памяти в Python

Оставшаяся свободная виртуальная память используется для хранения Python объектов.

Для управления этой памятью в Python используется специальный механизм, который называется **аллокатор**. Он используется каждый раз, когда нам нужно создать новый объект.

Механизм памяти в Python

Обычно в программах не оперируют большими объектами. Числа, строки и т.д., занимают не большой объем в расчёте на одно значение. Но создаются они часто. И это приводило бы к проблемам, если бы Python все такие вызовы транслировал в операционную систему. Системный вызов на выделение памяти — трудозатратен, зачастую связан с переходом в контекст ядра операционной системы.

Поэтому одна из главных задач **аллокатора Python** — **оптимизация количества системных вызовов.**

Механизм памяти в Python

Для больших объектов (больше 512 байт) Python выделяет память напрямую у ОС.

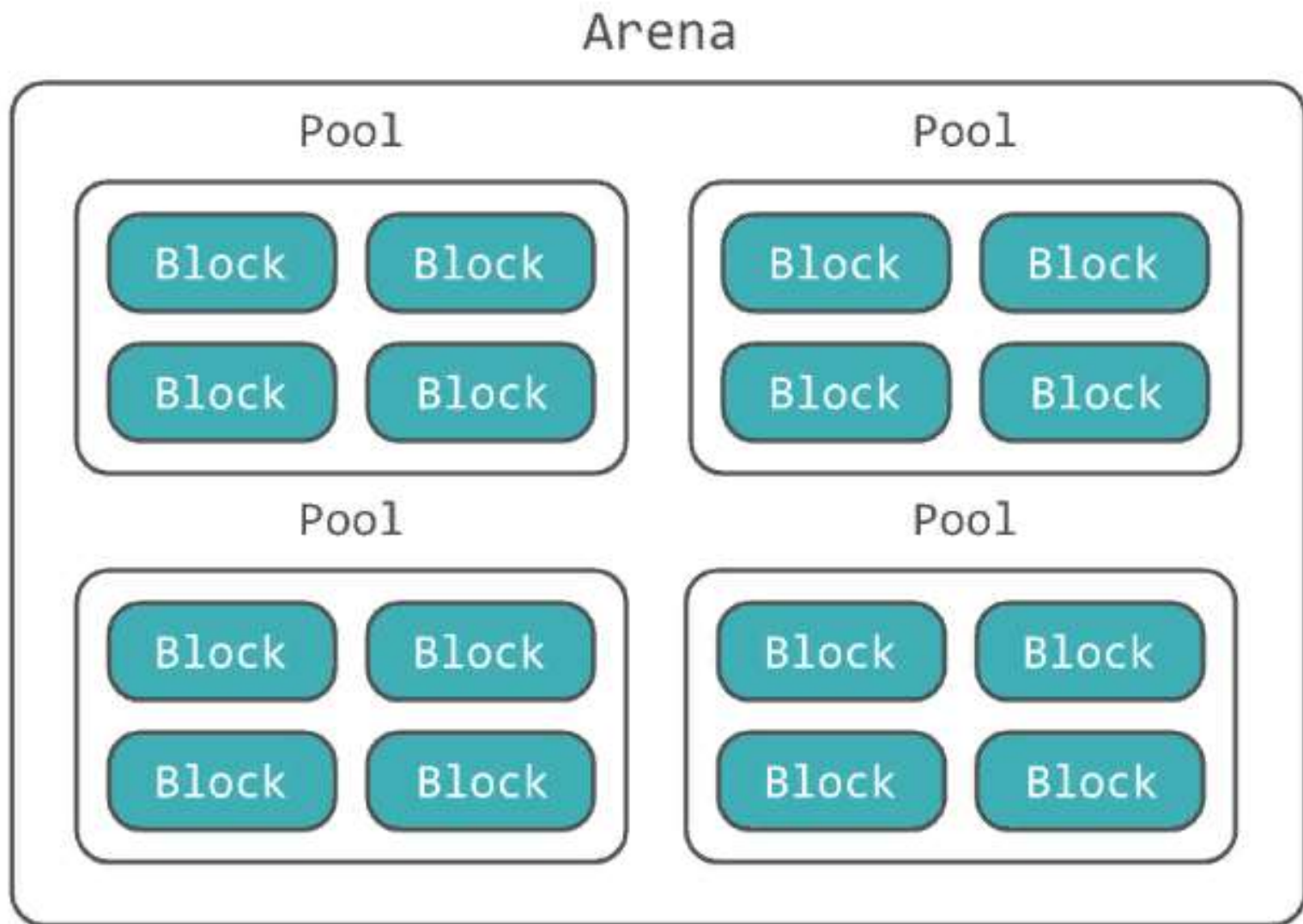
Обычно таких объектов не очень много в рамках программы, и создаются они нечасто. Поэтому расходы на создание таких объектов напрямую в оперативную память не высоки.

Механизм памяти в Python

Аллокатор для малых объектов (не больше 512 байт) использует три уровня абстракции:

- ❑ **блок** — кусок памяти, используемый для хранения одного объекта
- ❑ **пул** — кусок памяти, содержащий блоки (обычно одна страница виртуальной памяти)
- ❑ **арена** — большой непрерывный кусок памяти, содержащий пулы (обычно содержит несколько страниц виртуальной памяти)

Механизм памяти в Python



Блок

Блок — это кусок памяти, который может содержать только один Python объект фиксированного размера.

Размер блока может варьироваться от 8 до 512 байт и должен быть кратен восьми.

Все блоки в конкретном пуле имеют одинаковый размер и находятся в одном классе размера, который и определяет размер блока.

Блок

Градация блоков:

Запрос памяти в байтах	Размер блока в байтах	Индекс класса размера
1-8	8	0
9-16	16	1
17-24	24	2
25-32	32	3
33-40	40	4
41-48	48	5
...	...	...
505-512	512	63

Пул

Пулы состоят из блоков одного размера.

Каждый пул работает по принципу двухсвязного списка с другими пулами того же размера.

Ограничение пула фиксированным размером блоков помогает с **фрагментацией**. Если объект уничтожается, аллокатор может заполнить это пространство новым объектом того же размера. Поэтому алгоритм может с легкостью найти необходимое место для требуемого размера блока даже среди множества пулов.

Размер пула равен 4 килобайта.

Пул

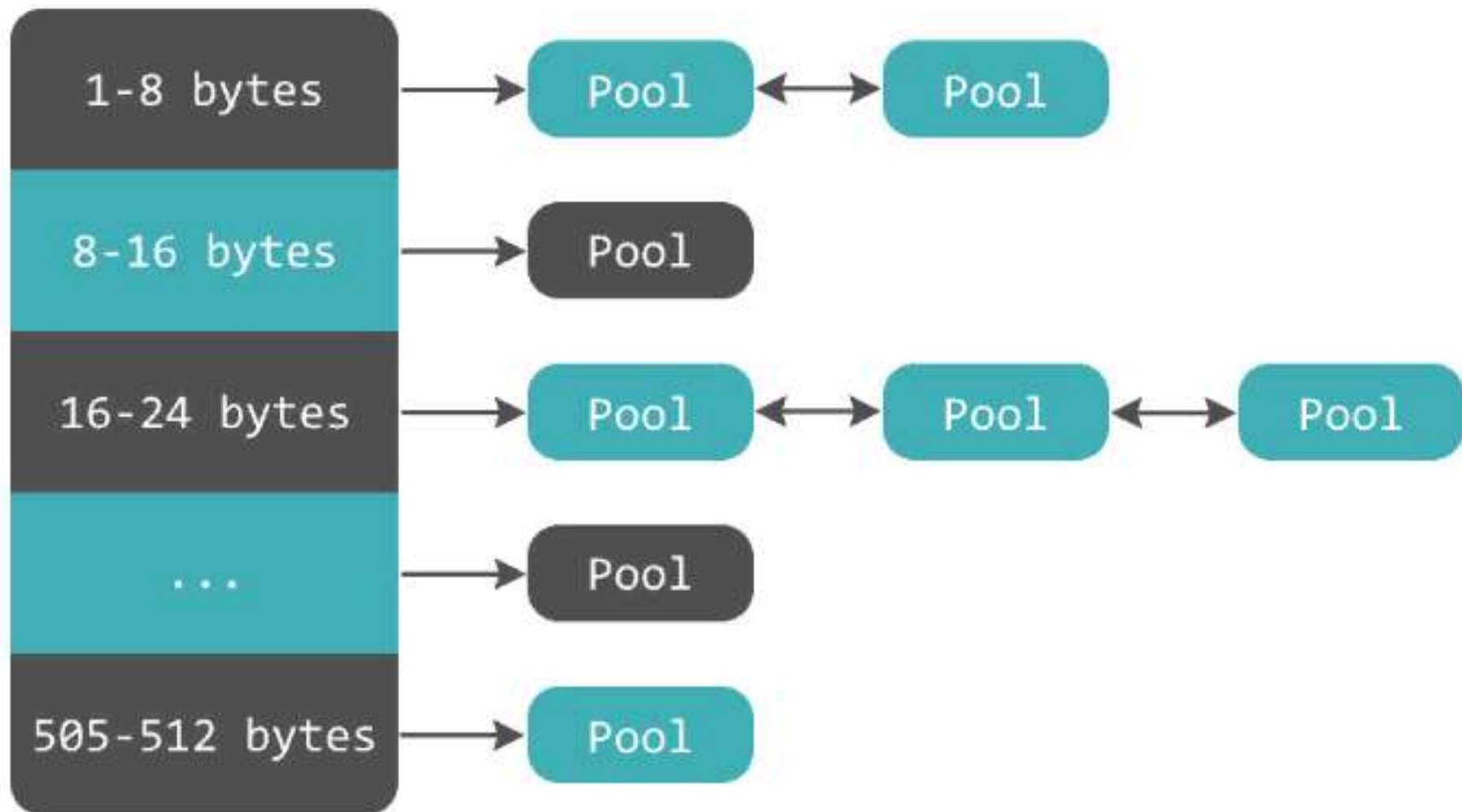
Для эффективного управления пулами Python использует дополнительный список, который называется **usedpools**. Он хранит указатели на пулы, сгруппированные по классам.

Все пулы с одинаковым размером блока связаны между собой. Чтобы перебрать их, нужно знать начало списка.

Если пулов такого размера нет, то при первом запросе памяти будет создан новый пул.

Пул

usedpools



Пул

Пулы находятся в трех состояниях:

- ❑ используемый (**used**) — частично заполненный, какие-то блоки заняты, какие-то свободны
- ❑ полный (**full**) — полностью заполненный, все блоки заняты
- ❑ пустой (**empty**) — все блоки свободны и доступны для записи (хранятся в списке пустых пулов **freepools**)

Пул

Используемый пул содержит блоки, в которые можно записать какую-то информацию.

Блоки **полного пула** все распределены и уже содержат данные.

Пустые пулы не содержат данных и могут быть разбиты на какие угодно классы размера при необходимости.

Пул

Например:

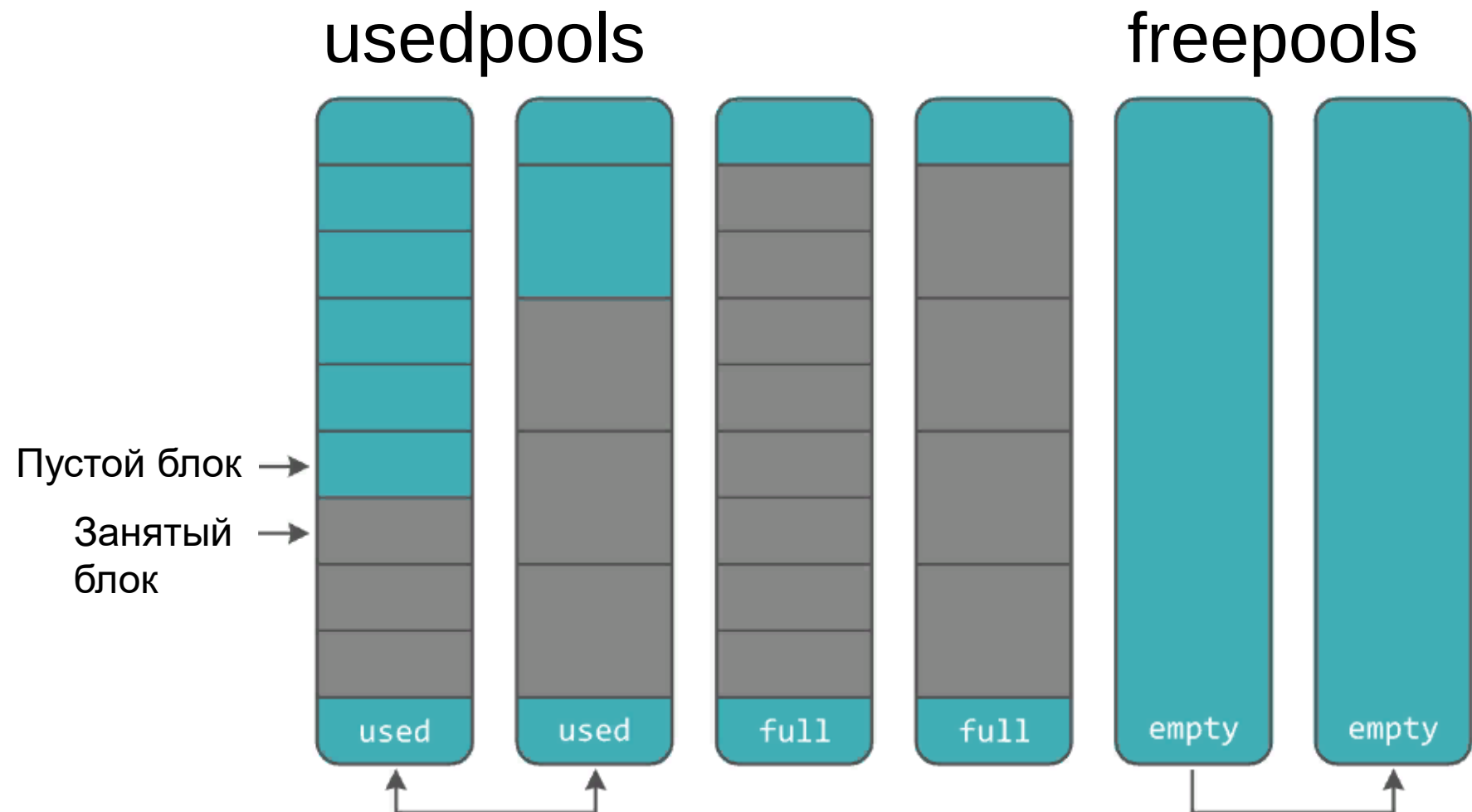
Коду необходимо 8 байт.

Если в списке используемых пулов нет пулов с классом размера в 8 байт, то новый пустой пул инициализируется, как хранящий блоки по 8 байт.

Затем пустой пул добавляется в список используемых пулов и может быть использован при следующих запросах.

Заполненный пул освобождает некоторые блоки, когда эта информация в них уже не нужна. Этот пул добавится в список используемых в соответствии со своим классом размера.

Пул

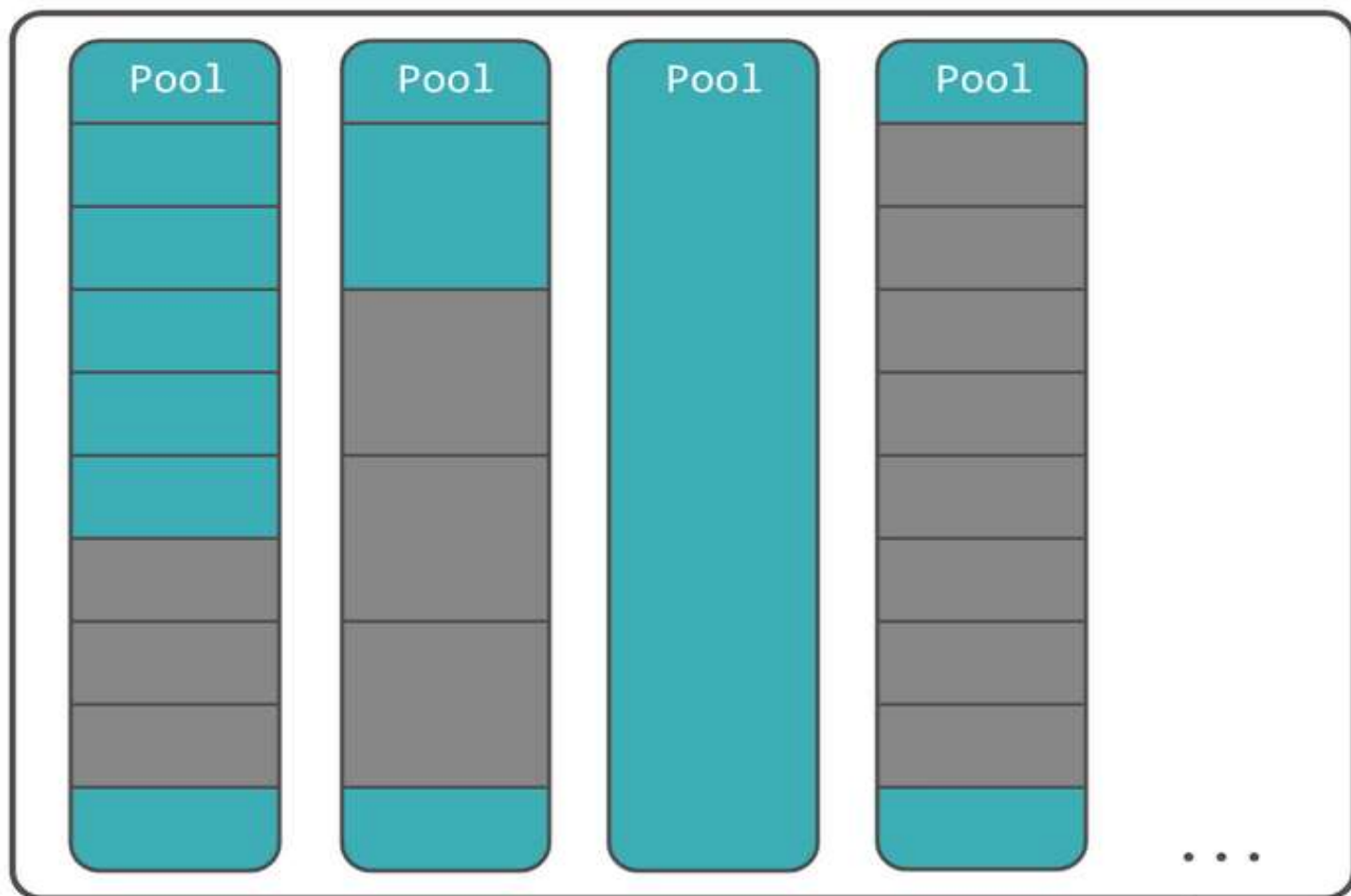


Пулы и блоки не выделяют память напрямую, они используют уже выделенное пространство в аренах.

Арена

Арена – это кусок памяти размером в 256 килобайт, который обеспечивает память для 64 пулов.

Arena



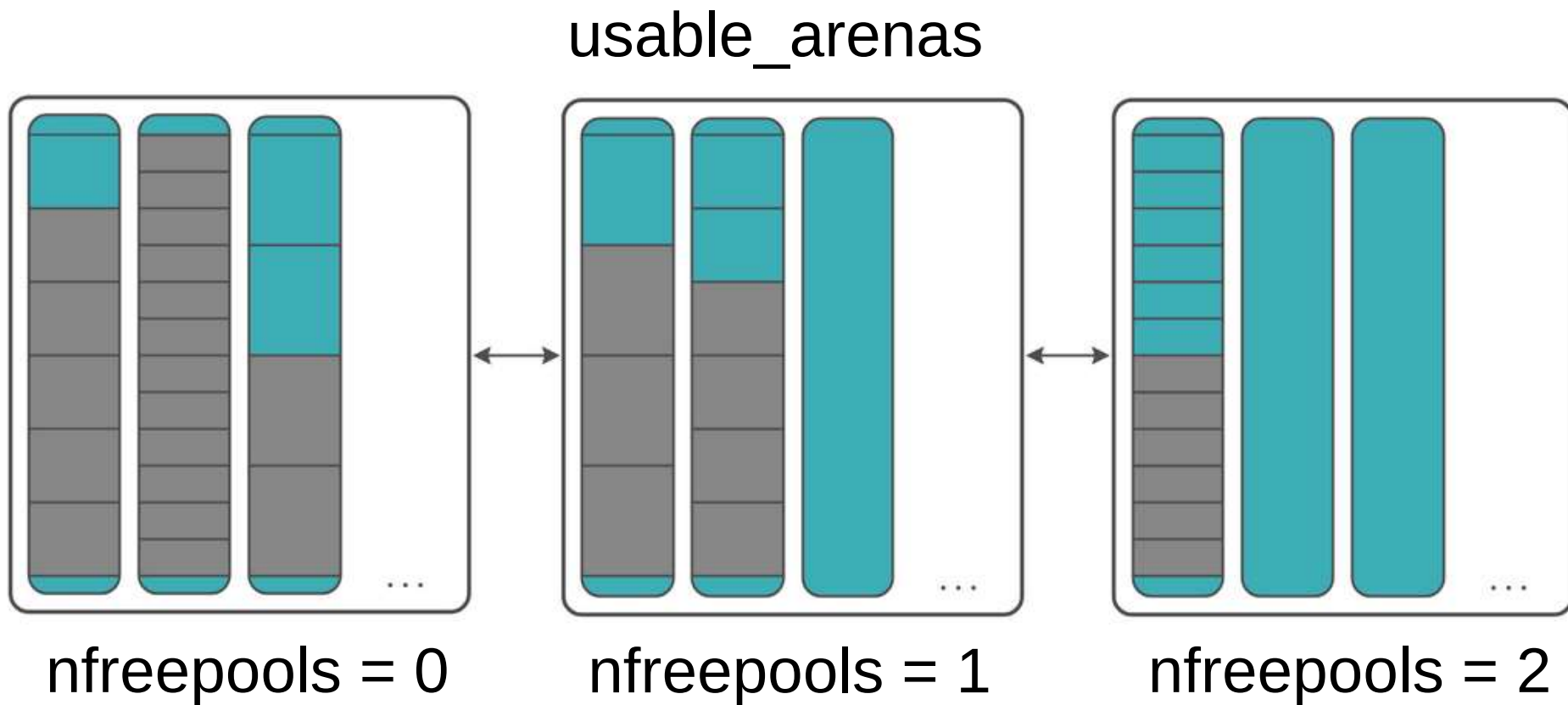
Арена

Арены, в отличие от пулов, не имеют явных разделений на состояния.

Они сами по себе организованы в двухсвязный список **usable_arenas**.

Этот список отсортирован по количеству свободных пулов. Чем меньше свободных пулов, тем ближе арена к началу списка.

Арена



Арена

Наиболее полная арена будет выбрана для записи еще большего количества данных. Такое расположение арен позволяет снизить объем потребляемой программой памяти.

При полном освобождении памяти Python возвращает память в пользование операционной системе.

Примечания

Примечание 1. Все вышесказанное относится к эталонной реализации Python, которая называется CPython. В других реализациях аллокатор может иметь отличия.

Примечание 2. Языки с ручным управлением памятью работают гораздо быстрее языков с автоматическим управлением памятью. Это цена, которую приходится платить за простоту написания программ.

Переменные в Python

Переменные в С и С++

Например код, который определяет переменную num:

```
int num = 101;
```

Исполнение этой строки кода проходит через несколько этапов:

- ❑ выделение достаточного количества памяти для числа;
- ❑ присвоение этому месту в памяти значения 101;
- ❑ отображение, что num указывает на это значение.

Переменные в С и С++

Упрощённо память
может выглядеть так:

num	
Location	0x7f1
Value	101

Здесь переменная `num` имеет виртуальный адрес `0x7f1` и значение `101`. Если позднее нам захочется изменить значение `num`, это можно сделать так:

```
num = 102;
```

Этот код присваивает переменной `num` новое значение `102`, перезаписывая предыдущее значение. Это означает, что переменная `num` **изменяема**.

Переменные в С и С++

Обновлённая
памяти для
значения:

схема
нового

num	
Location	0x7f1
Value	102

Расположение num в памяти не поменялось, поменялось только само значение. Это говорит о том, что num — это место в памяти, а не просто имя.

Можно сказать, что **переменная num владеет местом в памяти**. По сути, num — это пустая коробка, которая может содержать лишь одно целое число (тип int).

Переменные в С и С++

При этом создать новую переменную num1 можно следующим образом:

```
int num1 = num;
```

Этот код создаёт новую коробку под названием num1 и копирует в неё значение из num. Теперь схема памяти выглядит так:

num		num1	
Location	0x7f1	Location	0x7f5
Value	102	Value	102



Местоположение num1 — 0x7f5. Хотя в num1 и было скопировано значение num, однако переменная num1 владеет новым адресом в памяти.

Переменные в Python

В Python нет переменных, как таковых, вместо них имена. Можно использовать термин переменные, однако важно понимать разницу между переменными и именами.

Возьмём эквивалентный код из примера на C(C++) и напишем его на Python:

```
num = 101
```

Переменные в Python

Исполнение этой строки кода проходит через несколько этапов:

- ❑ создаётся специальный объект PyObject
- ❑ заполняется его поле Type, которое указывает на тип
- ❑ заполняется его поле Value, которое указывает на значение
- ❑ создаётся имя num
- ❑ имя num начинает указывать на созданный PyObject
- ❑ счётчик ссылок (поле Reference Count) объекта PyObject увеличивается на 1

Переменные в Python

Память может выглядеть таким образом:

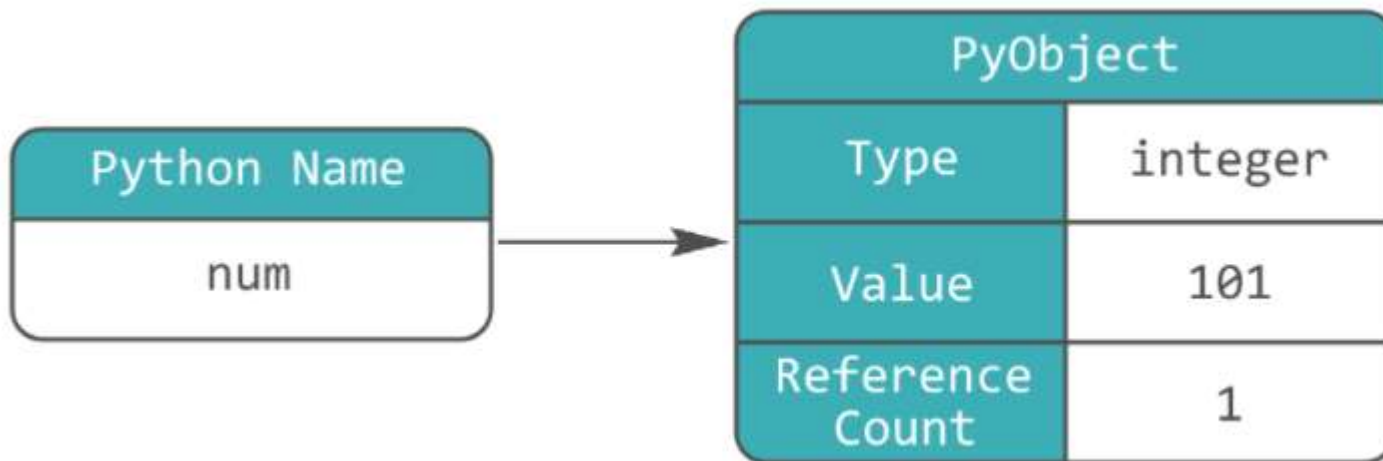


Схема памяти в Python отличается от схемы в С (С++). Вместо того чтобы num владел блоком памяти, в котором хранится значение 101, этой памятью владеет объект PyObject.

Таким образом, в Python имя num не владеет напрямую каким-либо адресом в памяти.

Переменные в Python

Для присвоения нового значения имени `num` используем следующий код:

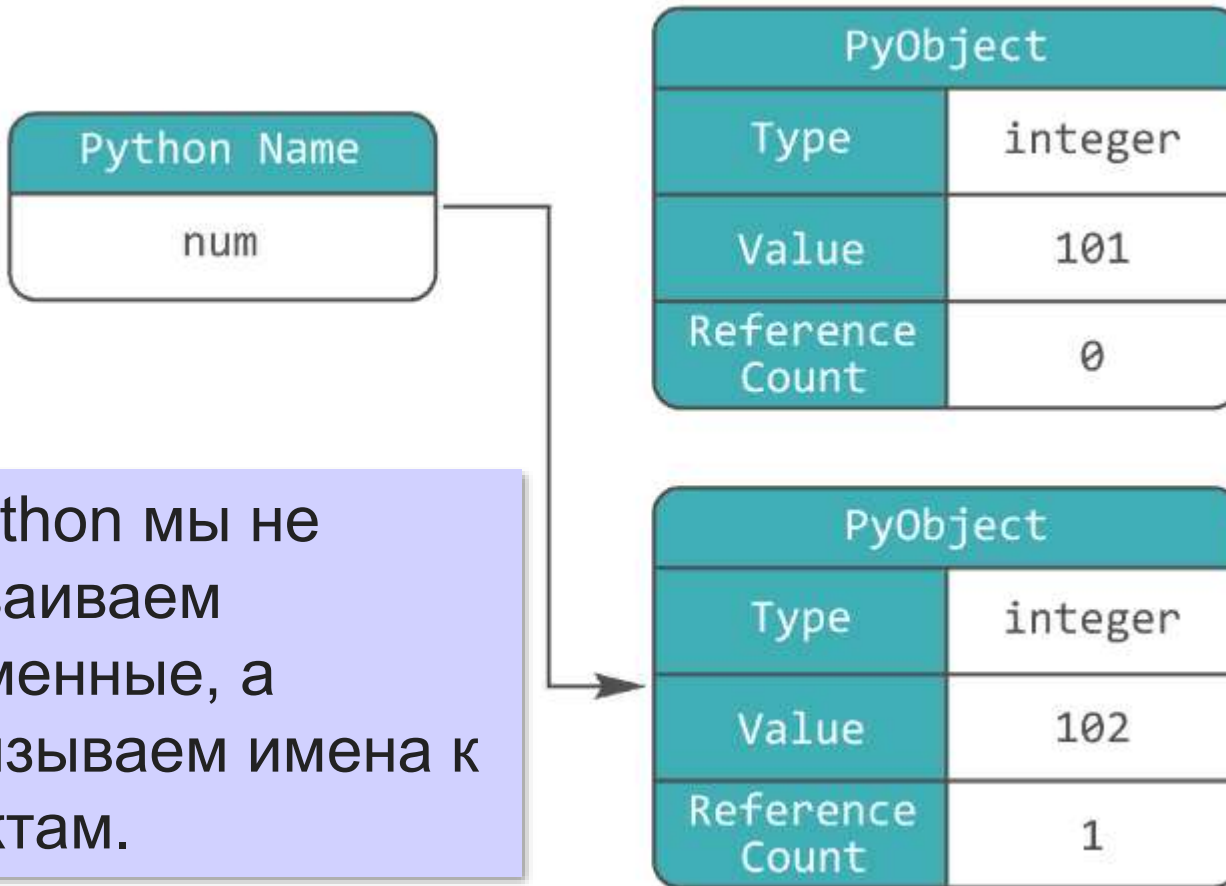
```
num = 101  
num = 102
```

Исполнение второй строки кода проходит через несколько этапов:

- ☐ создаётся новый `PyObject`
- ☐ заполняется его поле `Type`, которое указывает на тип
- ☐ заполняется его поле `Value`, которое указывает на значение
- ☐ имя `num` указывает на новый `PyObject`
- ☐ счётчик ссылок (поле `Reference Count`) нового `PyObject` увеличивается на 1
- ☐ счётчик ссылок (поле `Reference Count`) старого `PyObject` уменьшается на 1

Переменные в Python

Теперь схема памяти выглядит так:



! В Python мы не присваиваем переменные, а привязываем имена к объектам.

! Предыдущий PyObject теперь находится в памяти со счётчиком ссылок, равным 0, и подлежит удалению с помощью **сборщика мусора**

Переменные в Python

Мы можем ввести новое имя num1:

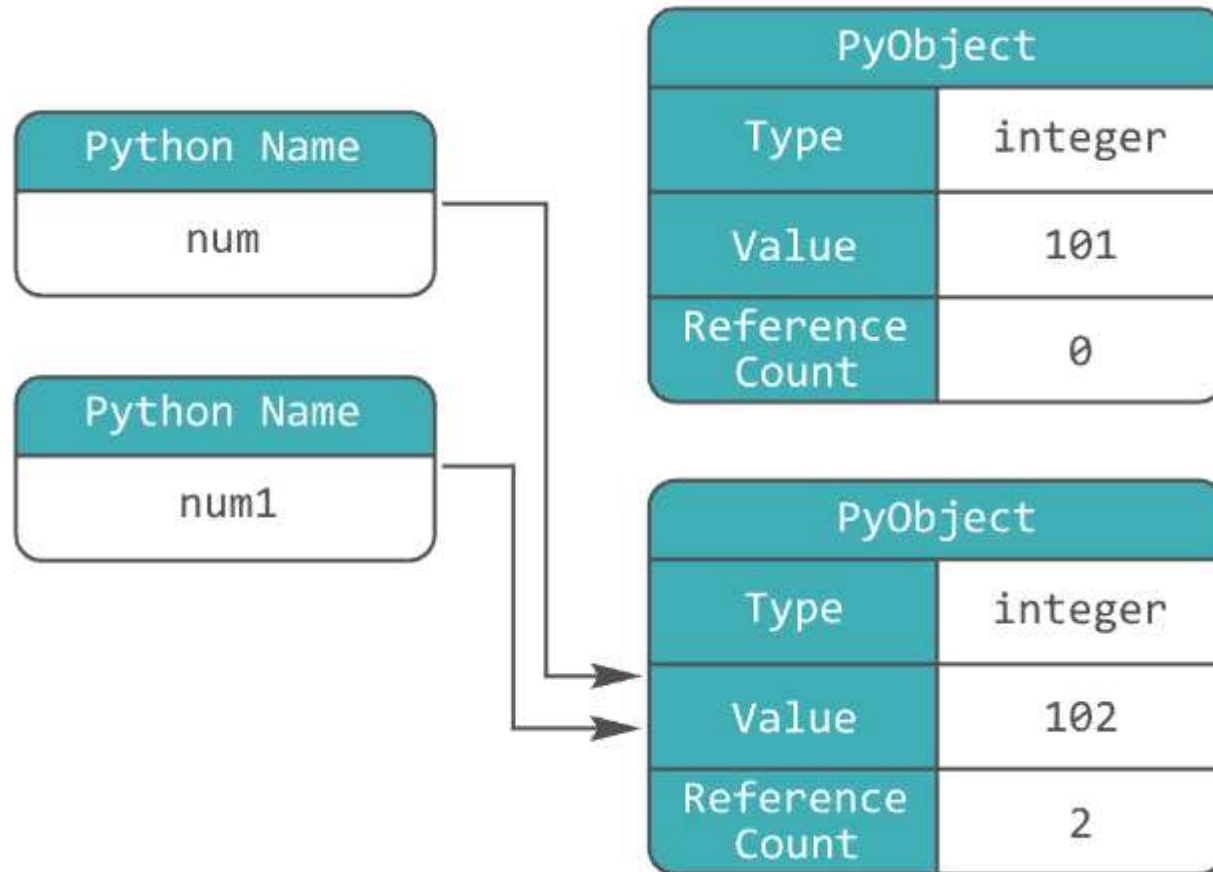
```
num = 101  
num = 102  
num1 = num
```



В памяти появится новое имя num1, но не новый объект!

Переменные в Python

Теперь схема памяти выглядит так:



Новый PyObject не создан, создано только новое имя, которое указывает на имеющийся объект, счётчик ссылок которого увеличился на 1.

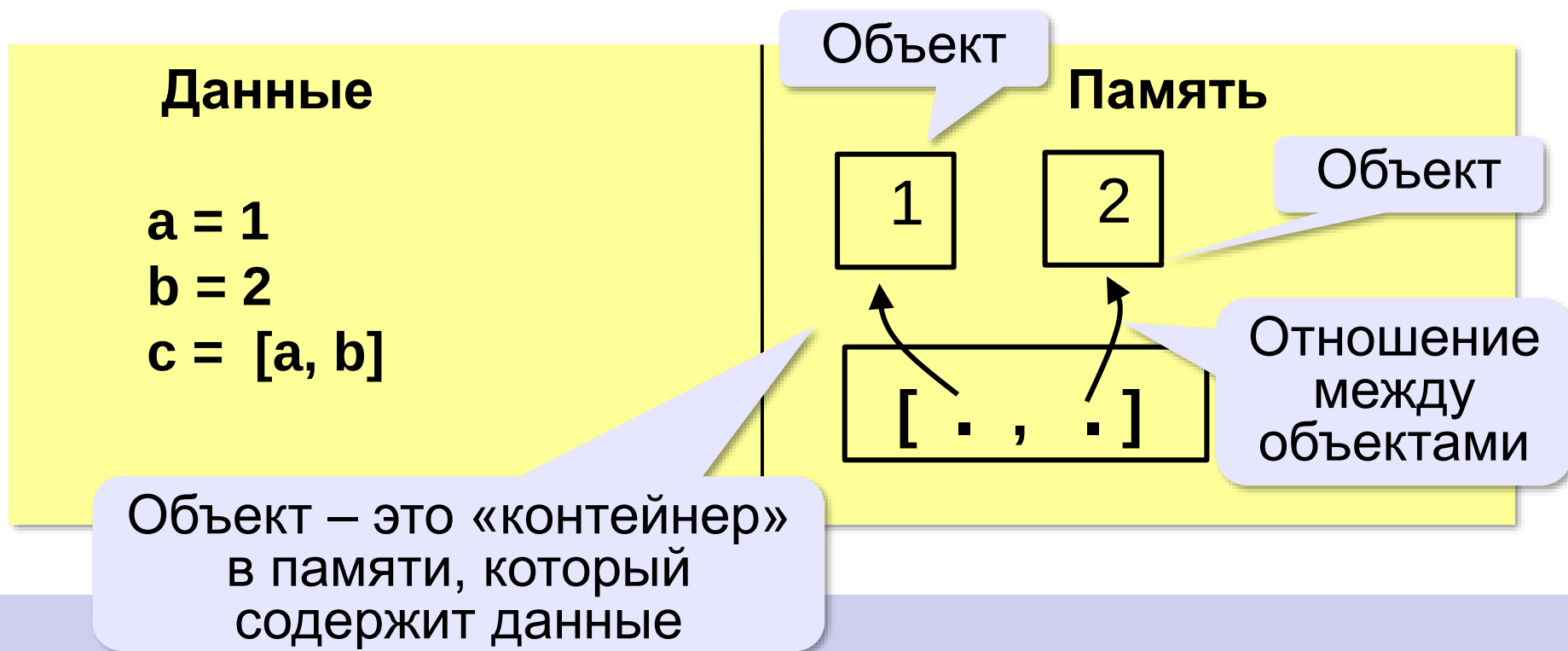
Объектно-ориентированное программирование. Язык Python

Объект в Python.

Объект в Python

В языке Python **все объект**: число, строка, список, кортеж, множество, словарь, даже функция

Данные – это объекты и отношения между ними



Объект в Python

Объекты в языке Python предоставляют универсальный способ хранения данных в памяти

Объекты отличаются друг от друга.

Любой объект имеет:

- Идентификатор;
- Тип;
- Значение

не может
измениться с
момента
создания
объекта



Идентификатор позволяет отличить один объект от другого. Гарантируется что никакие два объекта в один момент времени не обладают одинаковым идентификатором.

Объект в Python

`x = 4`

x
«ссылается»
на объект 4

- ▶ Создается объект 4
- ▶ Соответствие переменной x и идентификатора объекта 4

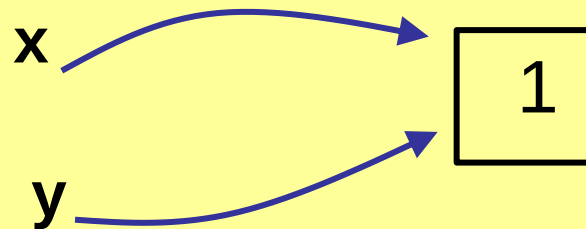


Оператор присваивания запоминает за именем переменной из левой части идентификатор объекта из правой части

Объект в Python

`y = x`

- ▶ Python "вспоминает" идентификатор объекта `x`
- ▶ Python запоминает соответствие переменной `y` и идентификатора объекта переменной `x`



Идентификатор объекта

Идентификатор объекта — это целое и постоянное число, которое никогда не изменяется после его создания.

В стандартной реализации Python (CPython) идентификатор объекта ассоциируется с адресом объекта в памяти.

Для того чтобы получить идентификатор объекта, используется встроенная функция **id()**.

Идентификатор объекта

Приведенный ниже код:

```
name = 'Timur'  
age = 29  
is_teacher = True  
print(id(name))  
print(id(age))  
print(id(is_teacher))
```



Получили идентификаторы объектов, созданных в разное время. Они не равны!

МОЖЕТ ВЫВОДИТЬ:

```
2017717924208  
2017661682800  
140729045662568
```



При каждом новом запуске программы идентификаторы будут разные.

Идентификатор объекта

Определить ссылаются ли переменные на один и тот же объект можно при помощи оператора `is`:

```
x = [1, 2, 3]
y = x
y is x           # True
y is [1, 2, 3]   # False
```

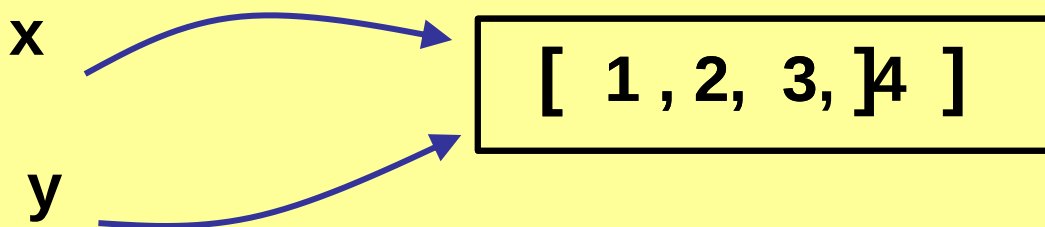
Если какая-либо другая переменная указывает на объект, который мы изменили, эта переменная также отразит это изменение, но не потому что изменилась переменная, а потому что изменился объект, на который она ссылается.

Идентификатор объекта



Изменяется объект, а не переменная!

```
x = [1, 2, 3]
y = x
print(y is x) # True
x.append(4)
print(x) # [1, 2, 3, 4]
print(y) # [1, 2, 3, 4]
```



Тип объекта

Тип объекта определяет:

- действия над объектом, «поведение» объекта
 - возможные принимаемые значения объекта
- ▶ Тип объекта не может быть изменен после создания объекта (как и идентификатор)
 - ▶ Узнать тип объекта можно с помощью функции `type()`

```
x = [1, 2, 3]
```

```
type(x) # output is "<class 'list'>"
```

```
type(4) # output is "<class 'int'>"
```

```
# here comes a mindblowing example
```

```
type(type(x)) # output is "<class 'type'>"
```



Все в Python - объект!

ИЗМЕНЯЕМЫЕ И НЕИЗМЕНЯЕМЫЕ ТИПЫ ДАННЫХ

Важно понимать какие значения могут принимать объекты того или иного типа.

Immutable object – неизменяемый объект

Mutable object – объект, который может изменить свое значение в течение жизни



Числа - не изменяемые объекты

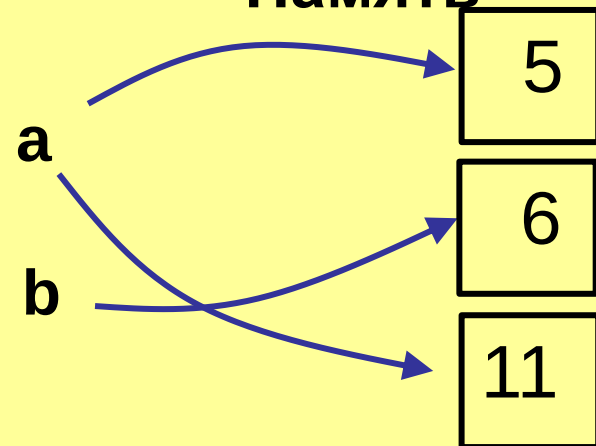
Данные

$a = 5$

$b = 6$

$a = a + b$

Память



ИЗМЕНЯЕМЫЕ И НЕИЗМЕНЯЕМЫЕ ТИПЫ ДАННЫХ

Например:

```
data = 'beegreek'  
data[0] = 'B'  
print(data)
```

? Что получим?

Результат:

TypeError: 'str' object does not support item assignment

? Почему?



Строки (тип str) являются
неизменяемыми

ИЗМЕНЯЕМЫЕ И НЕИЗМЕНЯЕМЫЕ ТИПЫ ДАННЫХ

Все строковые методы возвращают новые объекты, а не изменяют исходные.

Аналогичным образом работают методы других неизменяемых типов данных.

```
a = 'beegreek'  
b = a.upper() + a.lower()
```



Сколько Python объектов будет создано во время выполнения приведенного кода?

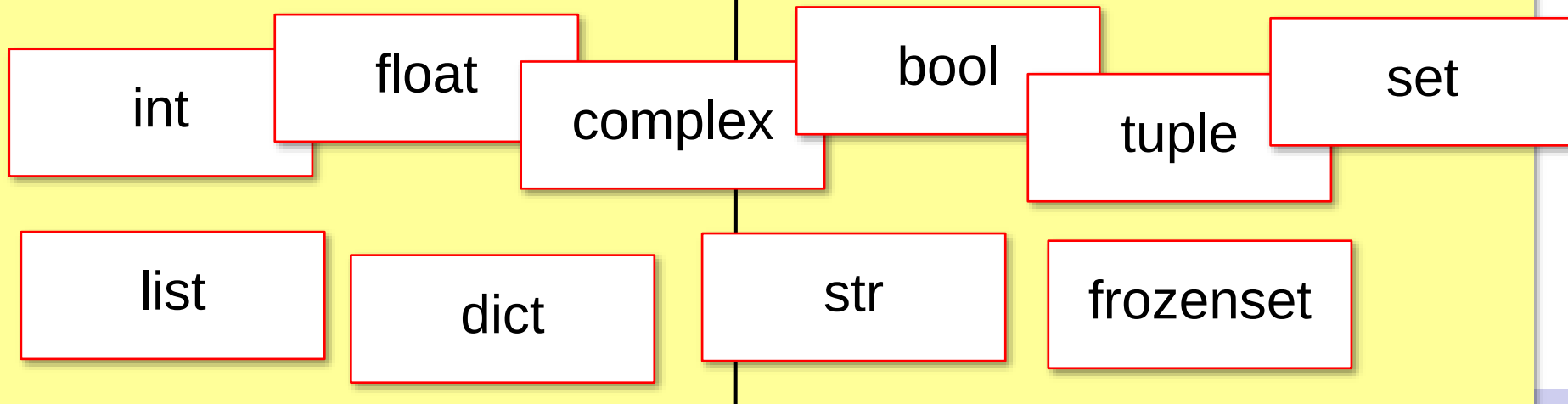
Будет создано 4
PyObject

ИЗМЕНЯЕМЫЕ И НЕИЗМЕНЯЕМЫЕ ТИПЫ ДАННЫХ

Возможность объекта изменять свои значения зависит от типа объекта:

Immutable object

Mutable object



Изменяемые и неизменяемые типы данных

Пример 1:

```
nums = [1, 2, 3]
print(nums)
print(id(nums))
nums = [1, 2, 3] + [4]
print(nums)
print(id(nums))
```



Не изменяем исходный список, а создаем новый и присваиваем его прежней переменной.

Результат:

```
[1, 2, 3]
2130653012352
[1, 2, 3, 4]
2130653007680
```



Данный код всегда выводит различные идентификаторы!

Изменяемые и неизменяемые типы данных

Пример 2:

```
nums = [1, 2, 3]
print(id(nums))
print(nums)
nums.append(4)
print(id(nums))
print(nums)
```



Единожды создаем список, а после изменяем его, добавляя в него новый элемент.

Результат:

```
2675065348480
[1, 2, 3]
2675065348480
[1, 2, 3, 4]
```



Данный код всегда выводит равные идентификаторы!

Изменяемые и неизменяемые типы данных

Пример 3:

```
x = [1, 2, 3]
y = x
y.append(4)
s = "123"
t = s
t = t + "4"
print(str(x) + " " + s)
```



Что получим?

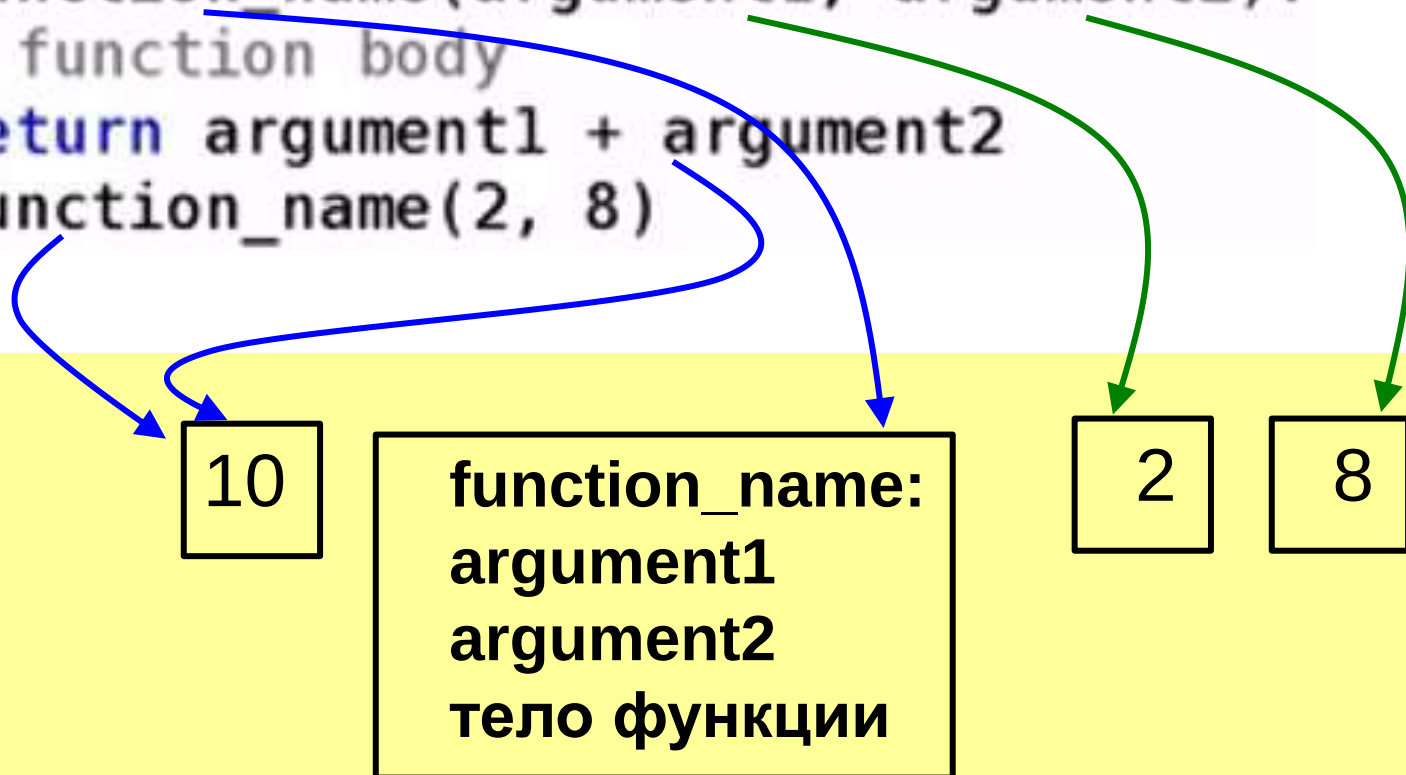
Результат:

[1, 2, 3, 4] 123

Функции как объекты

Функции в языке Python являются объектами.

```
def function_name(argument1, argument2):  
    # function body  
    return argument1 + argument2  
x = function_name(2, 8)
```



При каждом вызове функции Python инициализирует аргументы заново!

Пример:

```
a = []  
def foo(arg1, arg2):  
    a.append("foo")  
foo(a.append("arg1"), a.append("arg2"))  
print(a)
```



Что получим?

- ✓ a) ['arg1', 'arg2', 'foo']
- b) ['foo', 'arg1', 'arg2']
- c) ['foo']
- d) Код некорректен

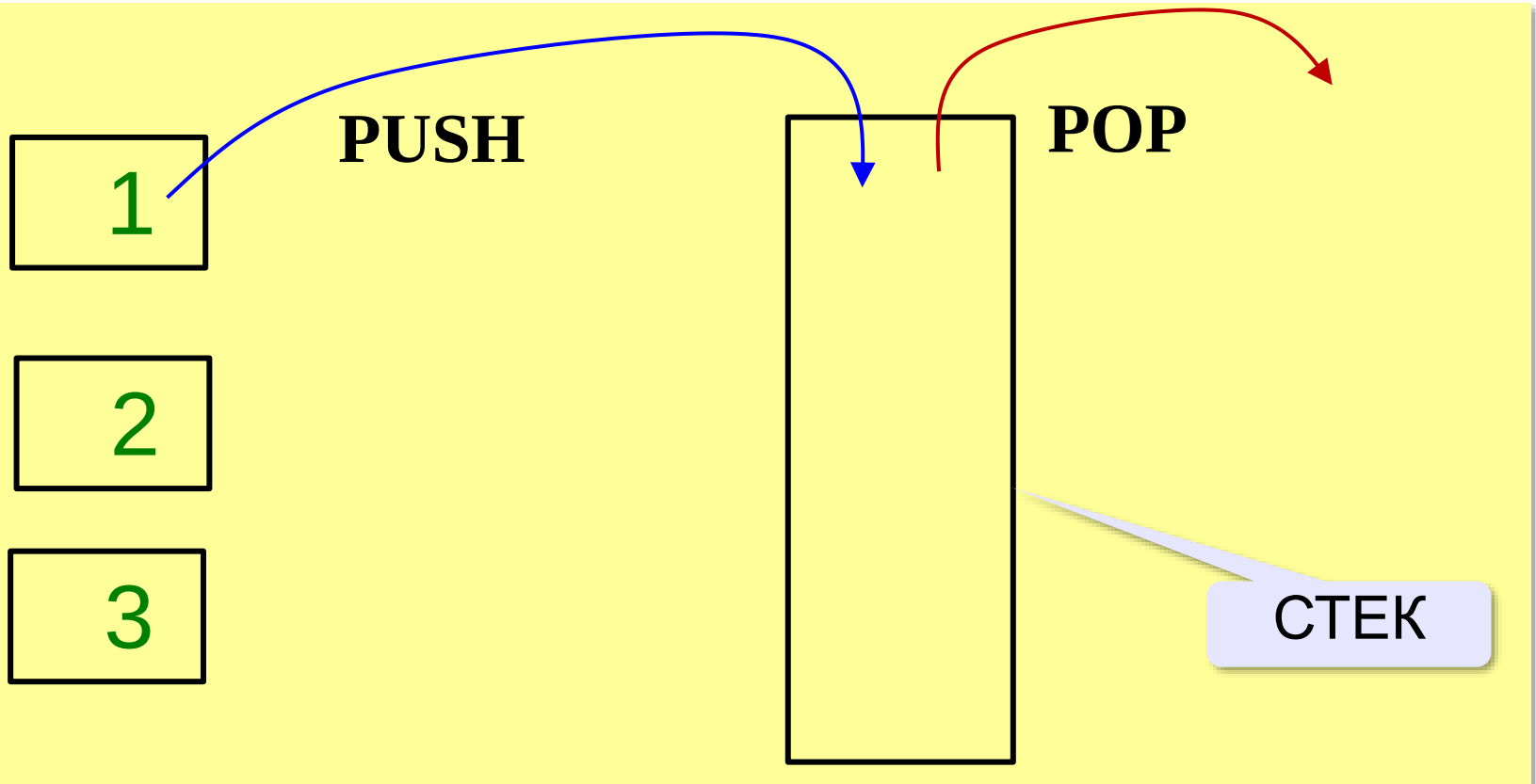
Стек

Важной концепцией в области исполнения кода и вызова функции является стек (машинный стек, стек исполнения, стек вызовов).

Стек — это структура данных, в которой элементы хранятся в порядке поступления. Его еще часто называют **LIFO** (Last-In/First-Out). Это отличает его от очереди, в которой элементы хранятся в порядке «первым пришел / первым обслужен» (**FIFO**).

Стек

Если бы в стеке хранились числа:



Операция, когда мы что-нибудь кладем на стек называется Push, а когда забираем элемент - Pop сверху!

не можем
ду
ирать только

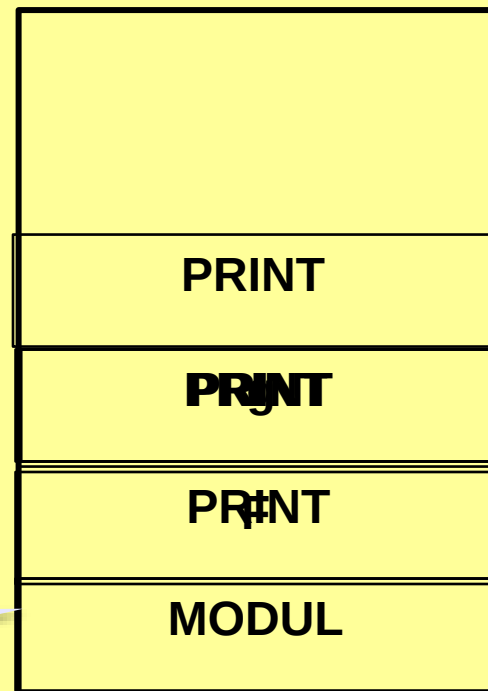
Стек вызовов функции

Стек вызовов хранит функции, которые вызваны:

```
def g():  
    → print("I am in function g")  
  
def f():  
    → print("I am in function f")  
    → g()  
    → print("I am in function f")  
  
→ print("I am outside of any function")  
→ f()
```

Все функции внутри стека вызовов исполняются. Та функция, которая лежит на верхушке стека исполняется реально, а остальные «ждут соседа сверху», пока он исполнится.

Стек вызовов



Объектно-ориентированное программирование. Язык Python

**Пространства имен
в Python.**

Пространства имен в Python (namespace)

Пространство имен—это совокупность определенных в настоящий момент символических имен и информации об объектах, на которые они ссылаются.

Существует 4 типа пространств имен:

- ☐ Встроенное.
- ☐ Глобальное.
- ☐ Объемлющее.
- ☐ Локальное.

Они обладают разными жизненными циклами. По мере выполнения программы Python создает необходимые пространства имен и удаляет их, когда потребность в них пропадает. Как правило, в любой момент времени существует множество пространств имен.

Встроенное пространство имен (builtins)

Встроенное пространство имен содержит имена всех встроенных объектов, которые всегда доступны при работе в Python.

Например, такие встроенные функции, как `max()` и `len()`, а также типы объектов— `int` и `str` и др.

При запуске интерпретатор Python создает встроенное пространство имен. Оно сохраняется до тех пор, пока интерпретатор не завершит работу.

Глобальное пространство имен (global)

Глобальное пространство имен содержит имена, определенные на уровне основной программы, и создаётся сразу при запуске тела этой программы. Сохраняется же оно до момента завершения работы интерпретатора.

Могут существовать и другие глобальные пространства имен. Интерпретатор также создает пространство данного типа для любого модуля, загружаемого программой при помощи выражения `import`.

Локальное (local) и объемлющее (enclosing) пространства имен

Интерпретатор создает новое пространство имен при каждом выполнении функции. Это пространство является локальным для функции и сохраняется до момента завершения ее действия.

Все эти пространства существуют до тех пор, пока выполняются соответствующие им функции. По завершении же этих функций Python может не сразу отозвать их из памяти, но при этом все ссылки на содержащиеся в них объекты сразу становятся недоступными.

Локальное и объемлющее пространства имен

Функции не существуют независимо друг от друга. Может быть, что одна функция определена внутри другой.

```
def f():  
    print('Start f()')  
    def g():  
        print('Start g()')  
        print('End g()')  
    g()  
    print('End f()')  
f()
```

Когда основная программа вызывает `f()`, Python создает для нее новое пространство имен. Аналогичным образом, когда `f()` вызывает `g()`, последняя получает свое собственное отдельное пространство. Пространство, созданное для `g()`, является **локальным**, а пространство, созданное для `f()`, — **объемлющим**.

Область видимости переменной

Наличие нескольких отличных пространств имен означает, что в процессе выполнения программы Python несколько разных экземпляров одного имени могут существовать одновременно. Пока каждый из них находится в собственном пространстве, все они обслуживаются по отдельности, и путаницы не происходит.



Предположим, что вы ссылаетесь на имя `x` в коде, а оно существует в нескольких пространствах. Как Python узнает, какое именно вы имеете в виду?

Область видимости переменной

Если ваш код ссылается на имя `x`, то Python будет искать его следующих областях видимости в таком порядке:

1) Локальная. Если вы ссылаетесь на `x` внутри функции, то интерпретатор сначала ищет его в самой внутренней области, локальной для этой функции.

2) Объемлющая. Если `x` не находится в локальной области, но появляется в функции, располагающейся внутри другой функции, то интерпретатор ищет его в области видимости объемлющей функции.

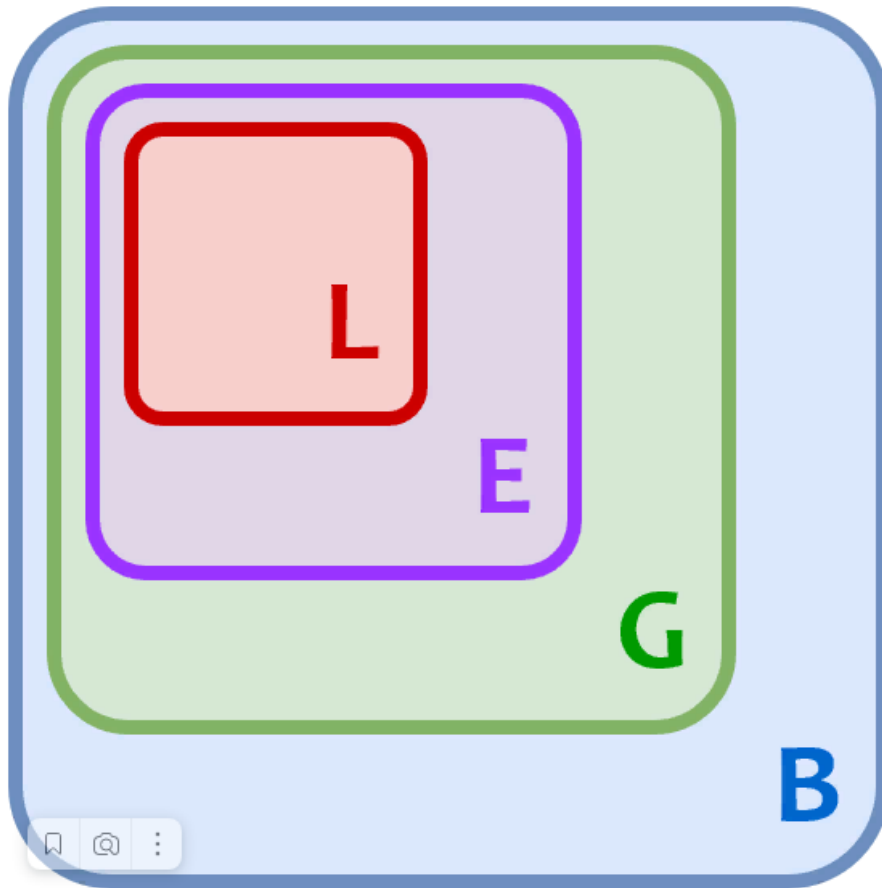
3) Глобальная. Если ни один из вышеуказанных вариантов не принес результатов, то интерпретатор продолжит поиск в глобальной области видимости.

4) Встроенная. Если интерпретатор не может найти `x` где-либо еще, то он направляет поиски во встроенную область видимости.



Эта последовательность составляет
суть **правила областей видимости LEGB**

Область видимости переменной



Если интерпретатор не находит имя ни в одной из этих областей, то Python вызывает исключение [NameError](#)

Объектно-ориентированное программирование. Язык Python

Класс в Python.

Класс в Python

Все **данные** в Python – это объекты и отношения между ними. У каждого объекта есть **тип**. Но не всегда удобно использовать только те типы, которые уже определены в языке.

Для того, чтобы определить свои собственные типы в языке Python есть **классы**.

Классы позволяют:

- ☐ Описать поведение объектов данного класса;
- ☐ Создать объекты данного класса.



Классы в языке Python - это механизм и синтаксис для описания собственных типов данных

Синтаксис определения класса в Python

Ключевое
слово

Имя класса

```
class MyClass:  
    a = 10
```

```
    def func(self):  
        print('Hello')
```

Тело класса

В отличие от функций тело класса выполняется в момент определения самого класса. Для тела класса создается отдельное пространство имен. И те имена, которые в этом namespace остались, затем закрепляются за объектом класса.

Синтаксис определения класса в Python

После того как объявление класса исполнилось, создается объект MyClass

```
class MyClass:
```

Атрибут

```
    a = 10
```

```
    def func(self):
```

Атрибут

```
        print( 'Hello' )
```

```
print(MyClass.a)
```

```
print(MyClass.func)
```

В данном примере, после исполнения класса имен в namespace остались имена: a и func. Они являются атрибутами объекта MyClass и мы сможем обратиться к ним, записав через точку.

Конструктор

Для создания объекта класса используется конструктор.

```
class MyClass:
    a = 10

    def func(self):
        print('Hello')

x = MyClass()
print(type(x))
# <class '__main__.MyClass'>
print(type(MyClass))
# <class 'type'>
```

Чтобы запустить конструктор класса и создать новый объект класса мы должны воспользоваться классом как функцией и вызвать ее.

Данная запись создает нам новый объект класса MyClass



У любого класса в языке Python всегда есть конструктор. Конструктор – это единственный механизм в котором создаются новые объекты!

Класс в Python

После создания класса в Python **гарантируется**:

- ❑ Всегда можно вызвать конструктор созданного класса:

Например:

```
x = MyClass()
```

- ❑ Мы можем обращаться к атрибутам класса:

```
MyClass.a
```

Действия над объектами, созданными с помощью конструктора

Определим пустой класс:

Экземпляр
класса
Counter

```
class Counter:  
    pass
```

Заменитель
функционала
класса

Создание
атрибута
экземпляра

```
Counter # class object  
x = Counter() # x is instance object  
x.count = 0  
x.count += 1
```

В данном случае в классе не определяется никаких методов или атрибутов. Однако поскольку в нем должно быть что-то определено, то в качестве заменителя функционала класса применяется оператор **pass**.

Конструктор

В языке Python есть возможность определить поведение конструктора. Это нужно для того, чтобы определить некоторые значения атрибутов уже созданных экземпляров класса по умолчанию.

Для того, чтобы определить поведение конструктора, необходимо вызвать функцию `__init__()` (по два прочерка с каждой стороны)

Конструктор

В качестве первого параметра конструктор принимает ссылку на объект - self.

```
class Counter:  
    def __init__(self):  
        self.count = 0
```

Конструктор, устанавливающий для каждого созданного экземпляра класса Counter атрибут count равный нулю

```
Counter # class object  
x = Counter()  
print(x.count) # 0  
x.count += 1
```

Функция `__init__()` не должна возвращать никакого значения, результат этой функции всегда объект **None**

Поведение установленных в конструкторе атрибутов не отличается от других атрибутов экземпляра класса. Можно менять их значение, обращаясь к ним в коде программы.

Конструктор с параметрами

Функция `__init__()` может принимать более одного аргумента:

```
class Counter:  
    def __init__(self, start=0):  
        self.count = start
```

```
Counter # class object  
x1 = Counter(10)  
x = Counter()  
print(x.count) # 0  
x.count += 1
```

Счетчик Counter
начинает свое
исчисление не
только с нуля, а с
любого
произвольного
значения

Например,
начальное
значение 10

Методы класса в Python

```
class Counter:  
    def __init__(self):  
        self.count = 0  
  
    def inc(self):  
        self.count += 1  
  
    def reset(self):  
        self.count = 0
```

Конструктор

Метод, увеличивающий
значение счетчика на 1

Метод, обнуляющий
значение счетчика

Метод – это процедура или функция, принадлежащая классу объектов.

Методы используются, чтобы объект изменить или что-то о нем узнать.

Методы класса в Python

```
class Counter:
    def __init__(self):
        self.count = 0

    def inc(self):
        self.count += 1

    def reset(self):
        self.count = 0
```

```
Counter # class object
x = Counter()
x.inc()
print(x.count)
Counter.inc(x)
print(x.count)
x.reset()
print(x.count)
```

Создание
экземпляра
класса
Counter

Вызов метода inc()
класса Counter

Вызов метода inc()
класса Counter

Вызов метода reset()
класса Counter

Пример:

Реализуйте класс Dog. У этого класса должен быть:

- конструктор, печатающий фразу «Родилась новая собака» в момент создания экземпляра класса Dog

```
class Dog:
    # Конструктор вызывается в момент
    # создания объекта
    # этого класса;
    def __init__(self):
        print("Родилась новая собака!")
# Создаем собаку (объект my_dog класса Dog)
my_dog = Dog()
```

Пример:

Реализуйте класс Cat. У этого класса должны быть:

- конструктор, принимающий параметры: имя и звук;

Объект класса Cat должен издавать звук "Мяу"

```
class Cat:
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound
    def make_sound(self):
        print(self.sound)
```

```
cat = Cat('Барсик', 'Мяу-у-у')
cat.make_sound()
```

Пример:

Реализуйте класс Points (точки). У этого класса должны быть

- конструктор, принимающий два числа x и y , координаты точки на плоскости;
- атрибуты x и y через которые можно будет получить координаты точки;
- метод `dist`, который принимает еще один объект класса Points и находит эвклидово расстояние между двумя точками.

```
class Points:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def dist(self, p2):
        distance = ((self.x - p2.x)**2 +
                    (self.y - p2.y)**2)**0.5
        return distance

p1 = Points(1.5, 1)
p2 = Points(1.5, 2)
print(p1.dist(p2))
```

Пример:



Какие числа будут выведены в результате выполнения данного кода?

```
class A:
    def __init__(self, val=0):
        self.val = val
    def add(self, x):
        self.val += x
    def print_val(self):
        print(self.val)
```

```
a = A()
b = A(2)
c = A(4)
a.add(2)
b.add(2)
a.print_val()
b.print_val()
c.print_val()
```

a) 2, 2, 0

b) 0, 0, 0

c) 2, 4, 0

✓ d) 2, 4, 4

e) 0, 2, 4

Объектно-ориентированное программирование. Язык Python

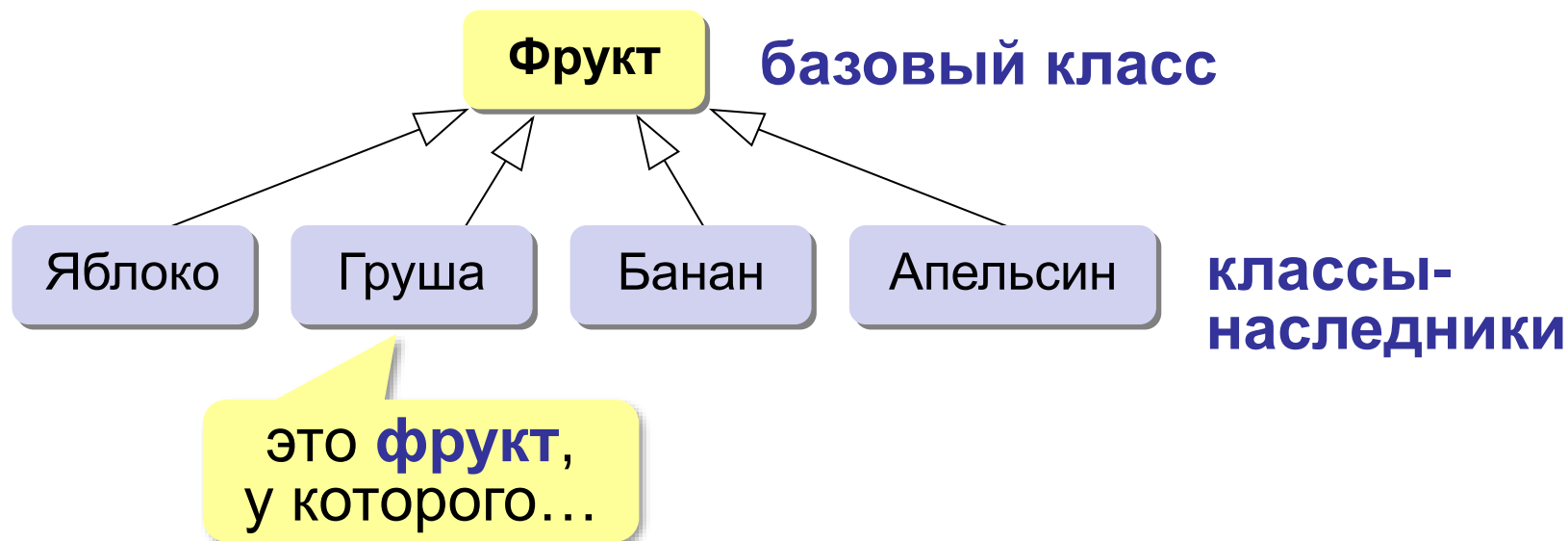
Свойства объектно-ориентированного подхода

Классификации



Что такое классификация?

Классификация – разделение изучаемых объектов на группы (классы), объединенные общими признаками.



Наследование

класс *Двудольные*
 семейство *Бобовые*
 род *Клевер*
горный клевер

наследует свойства
(имеет все свойства)

Класс Б является **наследником** класса А, если можно сказать, что Б – **это разновидность** А.

✓ яблоко – фрукт

яблоко – **это** фрукт

✓ горный клевер – клевер

горный клевер – **это**
растение рода *Клевер*

✗ машина – двигатель

машина **содержит**
двигатель (часть – целое)

Наследование в Python. Синтаксис.

Язык Python поддерживает наследование классов.

Наследование классов нужно тогда, когда мы хотим, чтобы объект вел себя как объект другого типа, но обладал некоторыми дополнительными атрибутами или методами.

Имя класса - наследника

Имя класса - родителя

```
class DerivedClassName(Base1, Base2, Base3):  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```


Наследование в Python.

Например, нужен объект, который будет «вести себя» как list, но при этом у него будет дополнительный метод, который определяет является ли число элементов в листе четным или нечетным. При этом мы не хотим создавать класс с нуля. В таком случае и используется наследование классов:

```
class MyList(list):
    def even_length(self):
        return len(self) % 2 == 0

x = MyList()
print(x) # []
x.extend([1, 2, 3, 4, 5])
print(x) # [1, 2, 3, 4, 5]
print(x.even_length()) # False
x.append(6)
print(x.even_length()) # True
```

Пример наследования

```
class Parent1:
    def func(self):
        print("Parent1")
class Parent2:
    def func(self):
        print("Parent2")
class Child(Parent1, Parent2):
    pass

obj = Child()
obj.func()
```



Результат?

Результат:

Parent1

Иерархия классов



Полиморфизм

Полиморфизм – это возможность классов-наследников по-разному реализовать метод с одним и тем же именем.

Обеспечивается иерархией наследования; Удобен в том случае, когда требуется одинаково работать с однотипными объектами;



Пример полиморфизма

```
class Pet:
    def __init__(self, name):
        self.name = name

class Cat(Pet):
    def make_sound(self):
        print(self.name, ' : Мяу')

class Dog(Pet):
    def make_sound(self):
        print(self.name, ' : Гав!')

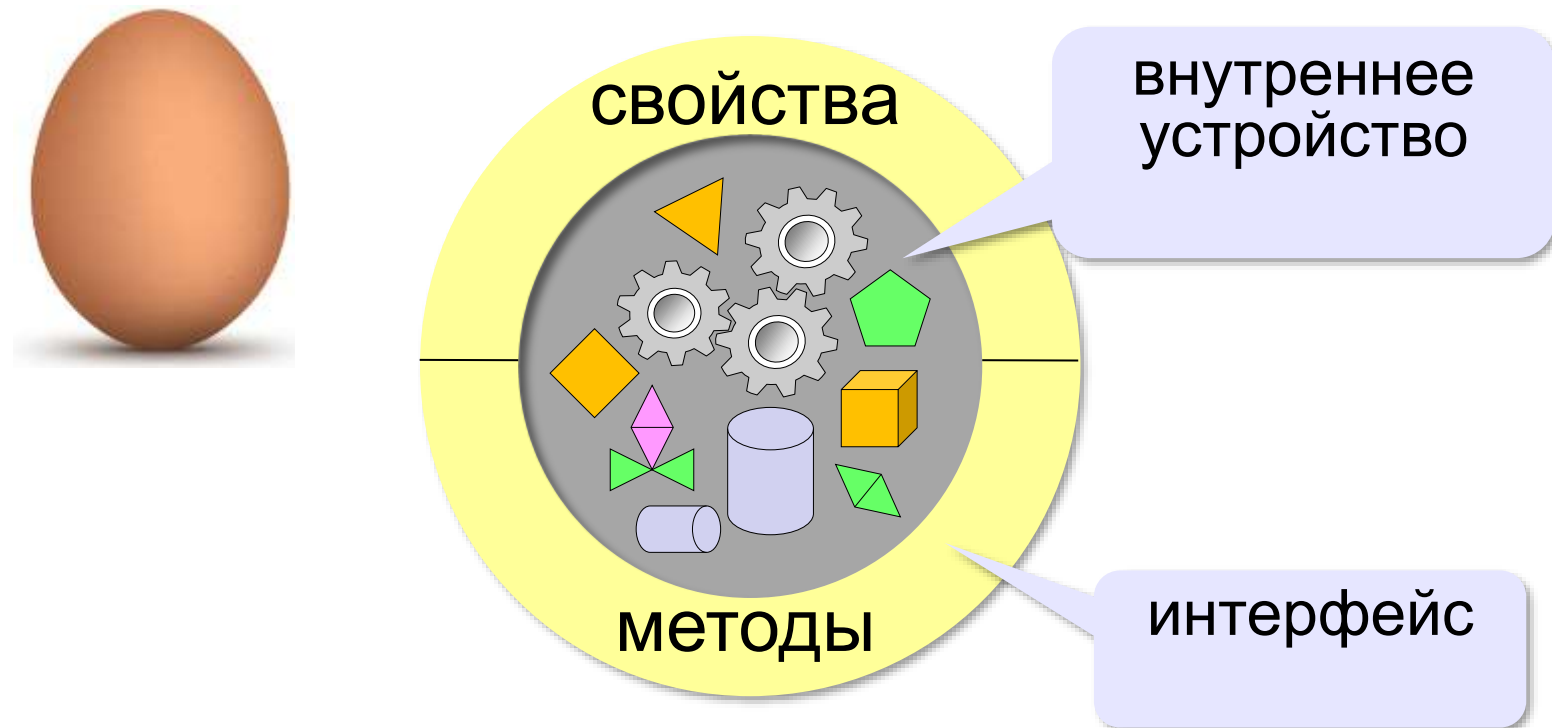
pets = Dog('Шарик'), Dog('Барбос'), Cat('Мурзик')
for pet in pets:
    pet.make_sound()
```

Результат
Шарик : Гав
Барбос : Гав
Мурзик : Мяу

Полиморфизм достигается за счет
переопределения методов класса.

Инкапсуляция

Инкапсуляция («помещение в капсулу»)



Инкапсуляция («помещение в капсулу») – скрывание внутреннего устройства объектов.

